

Embedded Linux & System Software Interview Handbook

Chapter 7 - Linux Kernel Synchronization & Concurrency (Part 4)

Topics: Atomic Operations, Memory Barriers and Lock-Free Synchronization

7.7 Atomic Operations

Atomic operations update shared variables as a single indivisible operation. They avoid race conditions for simple counters and flags without requiring spinlocks or mutexes.

Common Atomic APIs

API	Purpose
atomic_set()	Initialize an atomic variable
atomic_read()	Read value
atomic_inc()	Increment atomically
atomic_dec()	Decrement atomically
atomic_add()	Add value
atomic_sub()	Subtract value
atomic_cmpxchg()	Compare and exchange (CAS)

When to Use Atomic Operations

Atomic variables are ideal for reference counters, state flags and simple statistics. If multiple related variables must be updated consistently, use a lock instead.

7.8 Memory Barriers

Modern CPUs may reorder memory accesses for performance. Memory barriers enforce ordering constraints so that different CPUs observe memory operations in the intended sequence.

Barrier Types

Barrier	Purpose
mb()	Full memory barrier
rmb()	Read barrier
wmb()	Write barrier
smp_mb()	Barrier for SMP systems
smp_rmb()	Read barrier for SMP
smp_wmb()	Write barrier for SMP

Lock-Free Programming

Lock-free algorithms rely on atomic operations and compare-and-swap (CAS) rather than traditional locks. They improve scalability but require careful reasoning about memory ordering.

Common Mistakes

- Using atomic variables for complex data structures.
- Forgetting required memory barriers.
- Assuming operations are automatically ordered across CPUs.
- Mixing atomic and non-atomic accesses to the same variable.

Interview Questions

1. What is an atomic operation?
2. Atomic variable vs spinlock?
3. Explain compare-and-swap (CAS).
4. Why are memory barriers needed?
5. Difference between `mb()`, `rmb()` and `wmb()`?
6. When would you choose a lock-free algorithm?

Key Takeaways

Atomic operations are efficient for simple shared variables, while memory barriers ensure correct visibility across CPUs. Together they form the foundation of lock-free programming in the Linux kernel.